

A SUMMER INTERNSHIP REPORT
ON
XEBIA LEARNING MANAGEMENT SYSTEM

2-Month Summer Internship Report
submitted towards the partial fulfillment of the degree

BACHELOR OF TECHNOLOGY

BY
MRITUNJAI SINGH

23CS002773

Submitted to



Department of Computing and Informatics,
School of Engineering & Sciences,
Sir Padampat Singhania University,
Udaipur – 313601, Rajasthan, India

DECLARATION

I, Mritunjai Singh, student of B.Tech. (CSE), hereby declare that the 2-Month Summer Internship project report titled “Xebia Learning Management System”, submitted by me to the Department of Computing and Informatics, School of Engineering & Sciences, Sir Padampat Singhania University, Udaipur, Rajasthan, towards the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology, is an original record of the work carried out during my internship. The work embodied in this report has not previously formed the basis for the award of any degree, diploma, certificate, or other similar title or recognition.

Name and Signature of Student:

Mritunjai Singh

Place: Udaipur

Date: 11/08/2026

CERTIFICATE

This is to certify that the 2-Month Summer Internship project entitled “Xebia Learning Management System”, being submitted by Mritunjai Singh, submitted towards the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology, has been carried out under my supervision and guidance.

The matter embodied in this report has not been submitted, in part or in full, to any other university or institute for the award of any degree, diploma or certificate.

Biplove Singh

Xebia Trainee

Xebia

<https://xebia.com/>

Mrs. Chandini Joshi

Associate Professor

Department of Computer Science and Engineering,

Sir Padampat Singhania University, Udaipur

Dr. Amit Kumar Goel

Professor and Dean

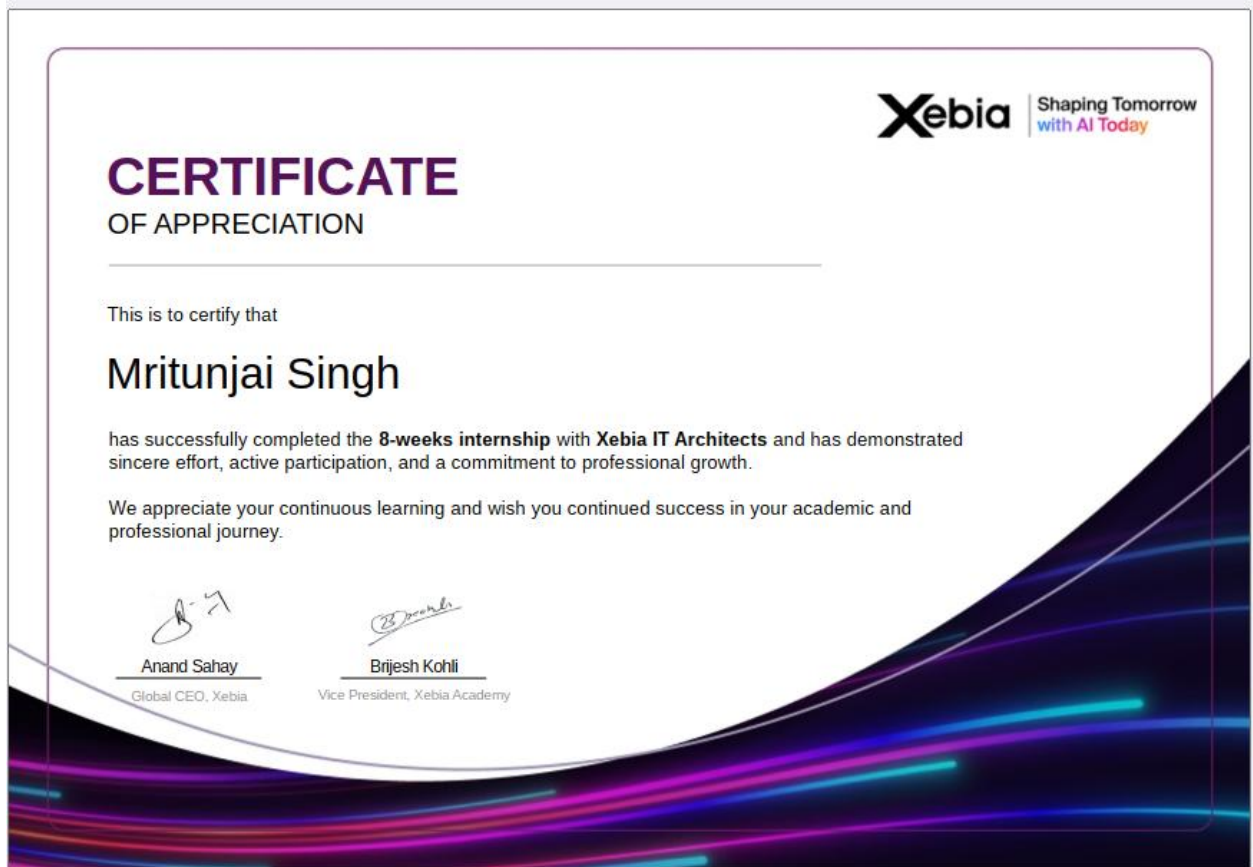
Department of Computing and Informatics,

School of Engineering & Sciences,

Sir Padampat Singhania University,

Udaipur, 313601, Rajasthan, India

COLORED COPY OF INTERNSHIP CERTIFICATE



ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my external guide, Mr. Biplove Singh, Xebia Trainee, for giving me the opportunity to work on the Xebia Learning Management System and for providing valuable guidance, encouragement, and support throughout the internship. His suggestions and professional insights helped me understand the practical aspects of developing an enterprise-level software application.

I am deeply thankful to my internal guide, Mrs. Chandini Joshi, Associate Professor, Department of Computer Science and Engineering, Sir Padampat Singhanian University, Udaipur, for her continuous guidance, constructive feedback, and academic support during the preparation and completion of this internship work. I also extend my sincere thanks to Dr. Amit Kumar Goel, Professor and Dean, Department of Computing and Informatics, School of Engineering & Sciences, Sir Padampat Singhanian University, for providing the institutional support and academic environment required to complete this internship successfully.

I gratefully acknowledge Xebia for providing an opportunity to gain practical exposure to modern full-stack development, enterprise application architecture, microservices, and collaborative software engineering practices. This project would not have been possible without the contribution, cooperation, and constant support of my team members. Their collaborative efforts, technical discussions, shared ideas, and assistance in different stages of the project played an essential role in bringing the Xebia Learning Management System to completion.

Finally, I would like to thank all faculty members, colleagues, friends, and my family for their motivation, patience, and encouragement throughout the internship period.

Mritunjai Singh

(23CS002773)

ABSTRACT

The Xebia Learning Management System is a comprehensive, enterprise-oriented web platform developed to centralize and streamline the planning, delivery, administration, and evaluation of organizational learning activities. Modern training environments require administrators, trainers, and learners to coordinate courses, batches, assessments, events, progress, and performance information through a consistent digital workflow. When these activities are handled through disconnected tools or manual records, they can lead to duplicated information, limited visibility, delayed evaluation, and difficulty in monitoring learning outcomes. The developed system addresses these challenges through a unified application that provides dedicated, role-focused portals while maintaining a common data and service architecture. Its objective is to support the complete learning lifecycle—from the creation of structured learning content and allocation of trainers to student participation, assessment submission, evaluation, and analytical review.

The frontend is implemented with React 19 and TanStack Router, supported by Tailwind CSS, reusable shadcn/Radix interface components, Zustand, React Context, React Query, Recharts, Framer Motion, and Monaco Editor. It provides three principal operational portals. The Admin Portal supports category and course management, hierarchical curriculum creation, user and event administration, batch and trainer-allocation workflows, assessment monitoring, and extensive analytical dashboards. The Trainer Portal enables trainers to create and manage batches, construct assessments containing multiple-choice, true/false, multi-select, descriptive, file-upload, and coding questions, publish them to selected batches, review submissions, perform manual or AI-assisted evaluation, export results, and monitor learner performance. The Student Portal allows learners to browse and enroll in courses, access structured course content, track completion progress, view assigned batches and events, register for events, take timed objective or coding assessments, inspect results, receive notifications, submit feedback, and obtain certificates.

The backend follows a microservices-based design consisting of a Spring Cloud API Gateway, five domain services for users, courses, batches, assessments, and events, and a shared common

library. Requests from the frontend pass through the gateway and are routed to the appropriate Spring service. Spring Data JPA and Hibernate provide persistence over PostgreSQL, while Flyway manages the dedicated course schema and Hibernate manages the remaining service tables. The system includes course enrollment and progress tracking, trainer allocation, assessment drafts and submissions, automated objective-question scoring, event registration with deadline and capacity validation, and aggregated administrative analytics. The present prototype uses email-based demonstration authentication, role-aware frontend routing, localStorage session persistence, and temporary identity headers; JWT utilities remain architectural scaffolding for future production security. The completed application demonstrates how a modular frontend, gateway-controlled APIs, independently organized services, and relational persistence can be combined into a scalable and maintainable enterprise learning platform. It delivers an integrated foundation for managing learning resources and measuring outcomes while retaining clear scope for stronger authentication, persistent real-time notifications, secure code execution, expanded automated testing, and further infrastructure hardening.

CONTENTS

INTRODUCTION	15
1.1 Background.....	15
1.1.1 Host Organization Context	15
1.1.2 Enterprise Learning Management Context	15
1.1.3 Problem Statement.....	16
1.1.4 Project Overview	16
1.1.5 Internship Context and Role	16
1.1.6 Principal Users and Their Needs.....	17
1.2 Objectives and Scope of Work	17
1.2.1 Primary Project Objectives	17
1.2.2 Functional Objectives by Portal.....	18
1.2.3 Technical Objectives.....	18
1.2.4 Personal Objectives and Responsibilities.....	18
1.2.5 Scope Included in the Completed Project	18
1.2.6 Scope Boundaries and Exclusions	19
1.2.7 Internship and Project Summary	19
LITERATURE REVIEW	20
2.1 Introduction to the Literature Review	20
2.2 Evolution and Purpose of Learning Management Systems.....	20
2.3 Enterprise LMS Stakeholders and Role-Oriented Design.....	20
2.4 User Experience, Accessibility, and Responsive Interaction	21
2.5 Component-Based Frontend Development	21
2.6 Routing and Information Architecture	21
2.7 RESTful APIs and Client–Server Separation.....	22
2.8 Microservices and the API Gateway Pattern.....	22
2.9 Relational Persistence, ORM, and Schema Management	22
2.10 Digital Assessment and Feedback.....	22
2.11 Learning Analytics and Decision Support	23
2.12 Authentication, Authorization, and Multi-Tenancy	23
2.13 Synthesis of the Reviewed Approaches	23
2.14 Research Gap and Project Positioning	24
2.15 Chapter Summary	24
ORGANIZATION / TOOLS & TECHNOLOGY OVERVIEW	25
3.1 Chapter Introduction.....	25
3.2 Organization Overview: Xebia	25
3.3 Internship Environment and Team Role	25
3.4 Overall Technology Architecture.....	25
3.5 Frontend Framework and Build Environment	26
3.5.1 React 19	26
3.5.2 TanStack Start, Router, and Query	26
3.5.3 Vite and Nitro	26
3.6 Styling and User Interface Technologies	27
3.6.1 Tailwind CSS and Design Tokens	27
3.6.2 shadcn/ui and Radix UI.....	27
3.6.3 Icons, Motion, and Notifications.....	27
3.7 Frontend State and Data Management	28
3.7.1 LMSContext	28

3.7.2 Zustand Administrative Store	28
3.7.3 Centralized API Service.....	28
3.8 Specialized Frontend Libraries	28
3.9 Backend Language, Framework, and Build Tools.....	29
3.9.1 Java Runtime and Compilation Target.....	29
3.9.2 Spring Boot and Spring Cloud.....	29
3.9.3 Maven and Lombok.....	29
3.10 Backend Service Technologies	29
3.10.1 Spring Web and REST Controllers.....	29
3.10.2 Spring Data JPA and Hibernate	29
3.10.3 Validation and Security Scaffolding	30
3.11 Microservices and Runtime Ports	30
3.12 Database and Schema Technologies	30
3.12.1 PostgreSQL 16.....	30
3.12.2 Flyway and Hibernate Schema Management.....	30
3.12.3 Redis and In-Memory Draft Storage.....	31
3.13 External Services and AI-Related Tools.....	31
3.14 Development Quality and Configuration Tools	31
3.15 Containerization and Deployment	31
3.16 Technology Selection Rationale	31
3.17 Chapter Summary	32
METHODOLOGY / SYSTEM DESIGN	33
4.1 Chapter Introduction.....	33
4.2 Development Methodology	33
4.2.1 Practical Work Phases.....	33
4.3 Requirements Analysis	34
4.3.1 Functional Requirements	34
4.3.2 Non-Functional Requirements	34
4.4 Actors and Use-Case Model	35
4.5 Overall System Architecture.....	35
4.6 Frontend System Design	36
4.6.1 Root and Portal Layouts.....	36
4.6.2 Route and Page Organization.....	36
4.6.3 Component Layers.....	36
4.6.4 State and Data Flow	37
4.7 Backend Service Design	37
4.8 API and Request-Flow Design.....	38
4.9 Database Design	38
4.10 Authentication and Tenant Design.....	40
4.11 Core Domain Workflow Design	40
4.11.1 Course and Curriculum Workflow.....	40
4.11.2 Batch and Allocation Workflow	41
4.11.3 Assessment Workflow	41
4.11.4 Event Workflow.....	41
4.12 Analytics Design.....	41
4.13 Error Handling and Validation Design	41
4.14 Deployment Architecture.....	41
4.15 Key Design Decisions and Trade-offs	42
4.16 Chapter Summary	42
IMPLEMENTATION.....	43
5.1 Chapter Introduction.....	43

5.2 Repository and Source Organization	43
5.3 Application Bootstrap, Session, and Shared Layout	44
5.4 Admin Portal Implementation.....	44
5.4.1 Dashboard, Categories, and Users	44
5.4.2 Course and Curriculum Management	44
5.4.3 Batch Management and Trainer Allocation	45
5.4.4 Administrative Assessment Management	45
5.4.5 Event and Analytics Hub Implementation	45
5.5 Trainer Portal Implementation.....	46
5.5.1 Trainer Dashboard and Batch Ownership	46
5.5.2 Assessment Builder.....	46
5.5.3 Evaluation, Leaderboard, and Reports	46
5.6 Student Portal Implementation	46
5.6.1 Dashboard, Batches, and Profile	46
5.6.2 Course Discovery, Enrollment, and Learning	47
5.6.3 Objective Assessment Attempt	47
5.6.4 Coding Assessment Workspace	47
5.6.5 Results, Certificates, Events, and Feedback.....	47
5.7 Frontend API and Context Implementation	48
5.8 Backend Implementation	48
5.8.1 API Gateway.....	48
5.8.2 User and Batch Services	48
5.8.3 Course Service	48
5.8.4 Assessment and Event Services	48
5.9 Database Entity Implementation	49
5.10 Representative End-to-End Implementation Traces.....	49
5.10.1 Course Enrollment and Progress	49
5.10.2 Assessment Submission and Evaluation	49
5.10.3 Event Registration.....	49
5.11 Personal Implementation Contribution	49
5.12 Chapter Summary	49
RESULTS AND DISCUSSION	50
6.1 Chapter Introduction	50
6.2 Overall Project Outcome.....	50
6.3 Results Against Project Objectives	51
6.4 Portal-Wise Results.....	51
6.4.1 Admin Portal Result.....	51
6.4.2 Trainer Portal Result	52
6.4.3 Student Portal Result	52
6.5 Frontend and User-Experience Results	52
6.6 Backend and Data Results	52
6.7 Assessment and Analytics Results	52
6.8 Deployment and Availability Results	53
6.9 Documented Load-Test Results	53
6.10 Discussion of Business and Learning Value	54
6.11 Limitations Observed in the Results	54
6.12 Internship Learning Outcomes.....	55
6.13 Chapter Summary	55
TESTING.....	56
7.1 Testing Objective and Approach.....	56
7.2 Test Environment and Tools.....	56

7.3 Page-by-Page Functional Testing	56
7.4 Portal Workflow and Integration Testing	57
7.5 Postman API Testing	57
7.6 Concurrent 50-Request Burst Testing.....	57
7.6.1 Load-Related Observations	58
7.7 Validation and Negative Testing.....	58
7.8 Database and Persistence Testing	58
7.9 Responsive, Accessibility, and Visual Testing	58
7.10 Security-Oriented Testing and Observations	58
7.11 Defects Found and Corrective Outcomes	59
7.12 Coverage Limitations	59
7.13 Testing Conclusion	59
CONCLUSION AND FUTURE SCOPE OF WORK	60
8.1 Conclusion	60
8.1.1 Achievement of the Internship Objectives	60
8.1.2 Personal and Professional Learning	60
8.2 Future Scope of Work.....	61
8.2.1 Production Authentication and Authorization.....	61
8.2.2 Persistent Notifications, Feedback, and Real-Time Updates	61
8.2.3 Secure Coding Assessment Infrastructure	61
8.2.4 Data and Schema Governance	61
8.2.5 Redis, Caching, and Resilience.....	61
8.2.6 Analytics and Reporting	61
8.2.7 Automated Testing and Quality Engineering.....	62
8.2.8 User Experience and Learning Features	62
8.2.9 Codebase Consolidation and Maintainability	62
REFERENCES	63
ANNEXURE - I.....	64
ANNEXURE - II	65

LIST OF FIGURES

Figure 1.1: High-level project context and stakeholder interaction	16
Figure 2.1: Evolution from content repositories to enterprise learning platforms	20
Figure 2.2: Quality attributes derived from the reviewed literature	23
Figure 3.1: Layered technology stack of the Xebia LMS	26
Figure 3.2: Frontend technology ecosystem and responsibilities.....	28
Figure 4.1: Iterative feature-oriented development cycle	33
Figure 4.2: Principal actors and high-level use cases.....	35
Figure 4.3: Overall frontend–gateway–service–database architecture.....	36
Figure 4.4: Frontend state ownership and data flow	37
Figure 4.5: Common backend request-processing layers.....	38
Figure 4.6: High-level entity groups and relationships	39
Figure 4.7: Detailed ER diagram of the Course Service schema	39
Figure 4.8: Detailed ER diagram of User, Batch, Assessment, and Event persistence	40
Figure 4.9: Cloud deployment topology	42
Figure 5.1: Principal frontend and backend source organization	44
Figure 5.2: Admin Portal implementation areas	45
Figure 5.3: End-to-end assessment lifecycle.....	47
Figure 6.1: Integrated functional outcomes of the project	51
Figure 6.2: Summary of the documented concurrent CRUD exercise.....	54
Figure 8.1: Proposed future development roadmap	62

LIST OF TABLES

Table 1.1: Primary users and their needs	17
Table 1.2: Summary of included project scope.....	19
Table 1.3: Internship and project overview.....	19
Table 2.1: Fragmented and integrated learning administration.....	21
Table 2.2: Literature insights and project implications.....	24
Table 3.1: Key technologies and their roles.....	26
Table 3.2: Backend services and local container ports	30
Table 3.3: Selected tools and current implementation notes.....	31
Table 4.1: Development phases and representative outputs.....	34
Table 4.2: Core quality requirements and current response.....	34
Table 4.3: Backend service boundaries and endpoint mappings.....	37
Table 4.4: Design decisions and trade-offs	42
Table 5.1: Assessment question types in the interface.....	46
Table 6.1: Project inventory and implementation scale	50
Table 6.2: Outcome against project objectives	51
Table 6.3: Deployment endpoints recorded in the project README.....	53
Table 7.1: Testing coverage and evidence	56
Table 7.2: Representative workflow and boundary tests	57
Table 7.3: Documented concurrent CRUD exercise.....	57
Table 7.4: Representative integration defects and fixes.....	59
Table 8.1: Final project outcomes.....	60
Table 8.2: Prioritized future-development roadmap	62

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, and Delete
CSS	Cascading Style Sheets
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
IAM	Identity and Access Management
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
ORM	Object-Relational Mapping
REST	Representational State Transfer
SEO	Search Engine Optimization
SQL	Structured Query Language
UI	User Interface
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

INTRODUCTION

1.1 Background

From 1 June to 31 July 2026, I completed a remote summer internship with Xebia as a Product Intern. I also served as Team Lead for our student project group. The central project was the Xebia Learning Management System (LMS), a web application intended to bring course setup, trainer and batch coordination, assessments, events, learner progress, and reporting into one shared platform. My individual contribution focused on frontend design and development; the complete product reflects the team's combined work.

1.1.1 Host Organization Context

Xebia is a technology consulting and software engineering organization with public areas of work that include digital transformation, cloud, data and AI, platform engineering, modern software delivery, and professional training. Its official locations information lists an office in Jaipur and identifies Gurugram as Xebia India's headquarters [1]. Because the internship was remote, this report does not describe it as an onsite placement at either location. The organization provides a relevant context for a learning platform: technology teams need ways to organize continuing education, practical training, and capability development.

1.1.2 Enterprise Learning Management Context

An enterprise LMS has to serve more than one kind of user. Administrators organize people, courses, batches, events, and reporting; trainers prepare learning activities and evaluate submissions; students need a clear route from course discovery to completion and feedback. The project reflects these needs through three portals—Admin, Trainer, and Student—within one application. Each portal has its own navigation, while the underlying course, batch, assessment, and event workflows are connected.

1.1.3 Problem Statement

When learning information is split across spreadsheets, forms, messages, and separate content stores, the same details can be entered more than once and records become difficult to compare. Students may not know which courses or assessments are current, trainers may struggle to track pending evaluation, and administrators may lack a consolidated view of participation. The project set out to reduce this fragmentation by linking the main learning activities in one system.

1.1.4 Project Overview

The LMS is a full-stack web application. Its React-based frontend contains 73 route definitions; 65 of these represent functional screens after layouts, redirects, and legacy shells are excluded. The backend is organized as one Spring API Gateway, five domain services (User, Course, Batch, Assessment, and Event), and a shared common library. PostgreSQL stores the principal records. The product connects course creation and enrollment with batches, assessment attempts, trainer evaluation, event registration, progress, and analytics. A coding workspace and AI-assisted features are present, but code execution is not a hardened production judge.

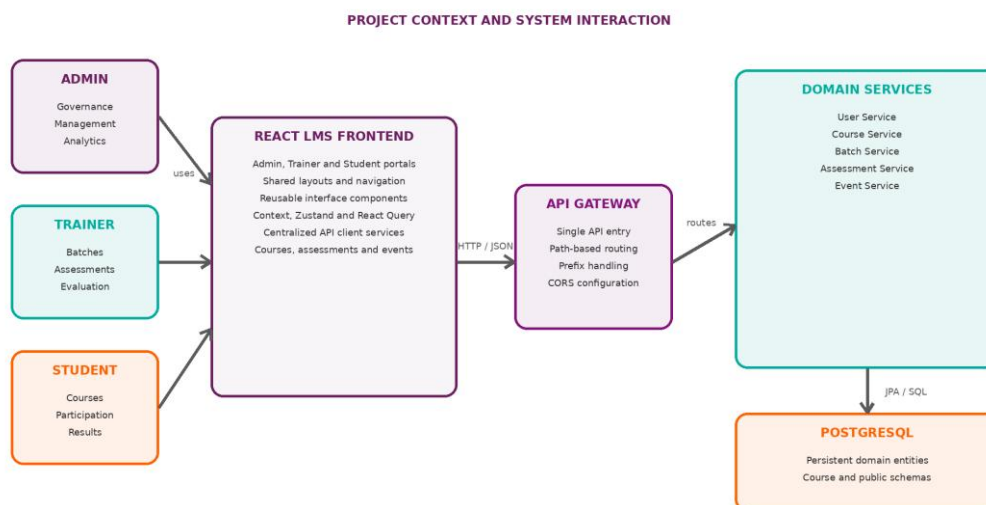


Figure 1.1: High-level project context and stakeholder interaction

1.1.5 Internship Context and Role

The work was completed online, so planning, review, and integration depended on clear communication and shared conventions. As Product Intern, I translated user needs into page

structure and interaction flows. As Team Lead, I helped maintain the group's direction and checked that separate contributions fit the same product. The main personal contribution was frontend design and development: navigation, layouts, reusable interface patterns, responsive behavior, and connections between pages, shared state, and available APIs.

1.1.6 Principal Users and Their Needs

The three user groups shaped the interface from the start. Administrators need breadth and oversight, trainers need efficient delivery and evaluation tools, and students need a focused learning path.

Table 1.1: Primary users and their needs

User group	Main need	Examples in the LMS
Administrator	Organize learning activity and review outcomes	Users, courses, batches, allocations, events, assessments, analytics
Trainer	Deliver training and evaluate learner work	Batch tools, assessment builder, grading, reports, notifications
Student	Find, complete, and receive feedback on learning	Course enrollment, progress, assessments, events, results, certificates

1.2 Objectives and Scope of Work

The project objective was to implement an integrated LMS prototype that supports the core activities of an enterprise training program. The internship also aimed to build practical experience in product analysis, frontend engineering, service integration, teamwork, and technical leadership.

1.2.1 Primary Project Objectives

- Bring the Admin, Trainer, and Student workflows into one product with role-focused navigation.
- Support structured courses, batch and trainer coordination, assessments, events, learner progress, and reporting.
- Connect the user interface to REST services and persistent relational data where the current backend provides those capabilities.
- Use shared layouts and reusable components to keep a large set of screens consistent.

- Prepare the application for local and cloud deployment through environment-based configuration and container definitions.

1.2.2 Functional Objectives by Portal

The Admin Portal manages categories, courses and curriculum, users, batches, allocations, events, assessment oversight, and analytics. The Trainer Portal focuses on owned batches, assessment creation, publishing, evaluation, leaderboards, and reports. The Student Portal supports course discovery and enrollment, lesson progress, assigned batches, event registration, objective and coding assessments, results, certificates, notifications, feedback, and profile information.

1.2.3 Technical Objectives

The technical objective was to separate presentation, API routing, domain logic, and persistence. On the frontend, nested portal layouts, reusable components, shared domain state, and a central API module help organize the feature set. On the backend, the gateway routes requests to services that follow controller, service, repository, and entity layers. PostgreSQL provides relational storage, with the current project using Flyway for the Course Service and Hibernate schema updates in several other services.

1.2.4 Personal Objectives and Responsibilities

My personal goals were to improve at turning product requirements into usable screens, deepen my React development skills, understand frontend-to-backend data flow, and coordinate a multi-member project. In practice, this meant reviewing navigation and page behavior, building and refining shared interface patterns, connecting actions to context or API methods, and helping team modules work together.

1.2.5 Scope Included in the Completed Project

The implemented scope covers the main learning lifecycle rather than every capability of a production training platform.

Table 1.2: Summary of included project scope

Area	Included work
Learning content	Categories, courses, curriculum hierarchy, content items, enrollment and progress
Coordination	Batches, student membership, trainer allocation and workload views
Assessment	Question configuration, attempts, objective scoring, trainer evaluation, results and exports
Participation	Events, registration, certificates, notifications and feedback screens
Platform	Role-based portals, API Gateway, five domain services, PostgreSQL and deployment configuration

1.2.6 Scope Boundaries and Exclusions

The active login is demonstrative: Trainer and Student access is matched by email and session data is stored in localStorage; Admin selection enters the portal without a verified identity. JWT and IAM-related classes are scaffolding, not an integrated security flow. Some requests use temporary user and tenant headers. Redis is configured, but assessment drafts currently use process-local memory. Notifications and feedback are not fully backed by dedicated persistent services, and the coding workspace does not provide isolated, secure code execution. These points are recorded as future work, not presented as completed production features.

1.2.7 Internship and Project Summary

Table 1.3: Internship and project overview

Attribute	Recorded detail
Internship period and mode	1 June 2026 to 31 July 2026; remote / online
Organization and role	Xebia; Product Intern
Team responsibility	Team Lead of the student project group
Primary individual contribution	Frontend design and frontend development
Project users	Admin, Trainer, and Student
Core outcome	Integrated enterprise-oriented LMS prototype

The project therefore combines product design and full-stack implementation. The chapters that follow describe the ideas behind the platform, the technology choices, the system design, the implemented workflows, and the evidence gathered during testing.

LITERATURE REVIEW

2.1 Introduction to the Literature Review

The LMS draws on research and practice in digital learning, user experience, web engineering, distributed services, data management, assessment, and analytics. This review focuses on ideas that influenced the project’s design and on the limits those ideas make visible.

2.2 Evolution and Purpose of Learning Management Systems

Learning platforms have grown from repositories for notes and announcements into systems that organize course structures, enrollment, assessment, feedback, communication, and reporting. Reviews of LMS use describe benefits such as flexible access and improved visibility, while also noting that outcomes depend on usable interfaces, good content, digital competence, infrastructure, and user support [3]. A platform’s feature count alone does not ensure that learners or trainers can use it effectively.

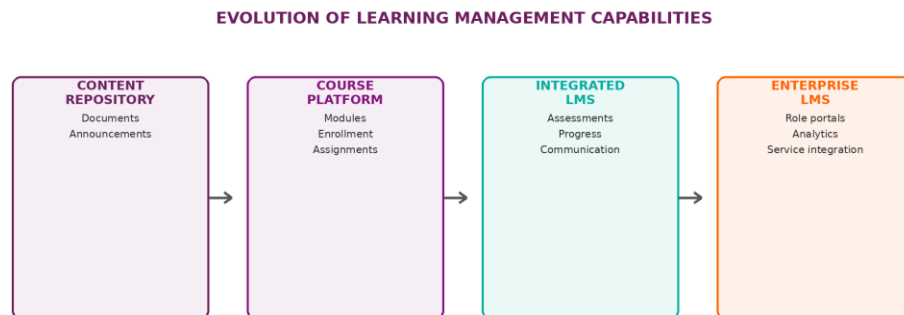


Figure 2.1: Evolution from content repositories to enterprise learning platforms

2.3 Enterprise LMS Stakeholders and Role-Oriented Design

Administrators, trainers, and students work with the same learning records but ask different questions of them. Separating their portals reduces irrelevant choices without splitting the product into unrelated tools. This is a usability decision, not a security guarantee: a production system must still enforce access rules on the server.

Table 2.1: Fragmented and integrated learning administration

Dimension	Fragmented approach	Integrated LMS approach
Learning content	Files and links spread across channels	Structured courses and curriculum
Assessment	Separate forms and manual consolidation	Attempts, submissions, scoring and review in one flow
Progress	Difficult to compare or continue	Progress and results linked to the learner
Oversight	Reports assembled from separate records	Shared dashboards and operational views

2.4 User Experience, Accessibility, and Responsive Interaction

A learner should be able to tell what to do next and whether an action succeeded. Consistent navigation, readable status labels, useful empty states, and feedback after changes all support that goal. WCAG 2.2 frames accessibility around perceivable, operable, understandable, and robust content [12]. The project uses responsive layouts and reusable controls, but formal keyboard, screen-reader, contrast, and conformance reviews remain necessary before claiming accessibility compliance.

2.5 Component-Based Frontend Development

React's component model supports building an interface from reusable parts [5]. In the LMS, shared cards, dialogs, tables, form controls, badges, and layouts reduce repeated work and help the three portals feel related. Domain components such as assessment cards and course content blocks sit above general UI primitives. Reuse is most useful when a component has a clear job; very broad components can become difficult to maintain.

2.6 Routing and Information Architecture

Nested and file-based routing helps keep portal layouts separate from individual pages. TanStack Router supports dynamic resource routes and nested layouts that suit courses, batches, assessments, and results [6]. The current code contains legacy and structural routes as well as functional screens, so route generation still needs routine review and automated navigation checks.

2.7 RESTful APIs and Client–Server Separation

REST separates the browser from server implementation through resource-oriented HTTP operations. Fielding's architectural work describes constraints including client–server separation,

stateless requests, and layered systems [9]. A shared frontend API module applies common headers, parsing, and error handling; named methods make page behavior easier to trace. Stable request and response contracts still require validation and documentation.

2.8 Microservices and the API Gateway Pattern

Microservices can align code ownership with domains, but introduce network failures, deployment work, and more difficult debugging [7], [8]. The LMS uses services for users, courses, batches, assessments, and events, with Spring Cloud Gateway providing one browser-facing route. The gateway currently routes requests and centralizes CORS; production concerns such as verified identity, rate limits, resilience policies, and end-to-end observability are future work [13].

2.9 Relational Persistence, ORM, and Schema Management

Courses, curriculum, enrollments, assessments, answers, events, and registrations have strong relationships, making PostgreSQL a reasonable fit [10]. JPA and Hibernate reduce repetitive persistence code, but explicit mappings, constraints, query review, and controlled schema changes remain important. The repository uses Flyway migrations in the Course Service and Hibernate update behavior in several other services; a single versioned migration policy would be easier to reproduce.

2.10 Digital Assessment and Feedback

Digital assessments combine question configuration, timing, attempts, submission, scoring, manual review, and result publication. Objective questions can be scored automatically when rules are explicit; descriptive and uploaded responses need human judgment. Coding tasks add another layer: a production judge requires isolated execution and resource limits. The project offers a coding workspace and AI-assisted feedback, but neither should be confused with a secure deterministic judge.

2.11 Learning Analytics and Decision Support

Dashboards can help staff interpret enrollment, progress, scores, pass rates, workload, and trends, but the metric definitions must be clear. Recent work on analytics-enhanced LMS experiences

notes potential for personalization alongside challenges in integration, interpretation, privacy, and user trust [4]. The LMS derives several views from shared data; for larger datasets, backend aggregations would be more suitable than processing full arrays in the browser.

2.12 Authentication, Authorization, and Multi-Tenancy

Identity confirms who a user is; authorization controls what that user can do; tenant isolation limits which organization's data can be accessed. Frontend route checks improve navigation but cannot protect an API from a direct request. JWT guidance emphasizes signature and algorithm validation, expiry checks, strong keys, and safe transport [11]. The repository contains JWT and tenant scaffolding, while the active login remains email-based demonstration authentication.

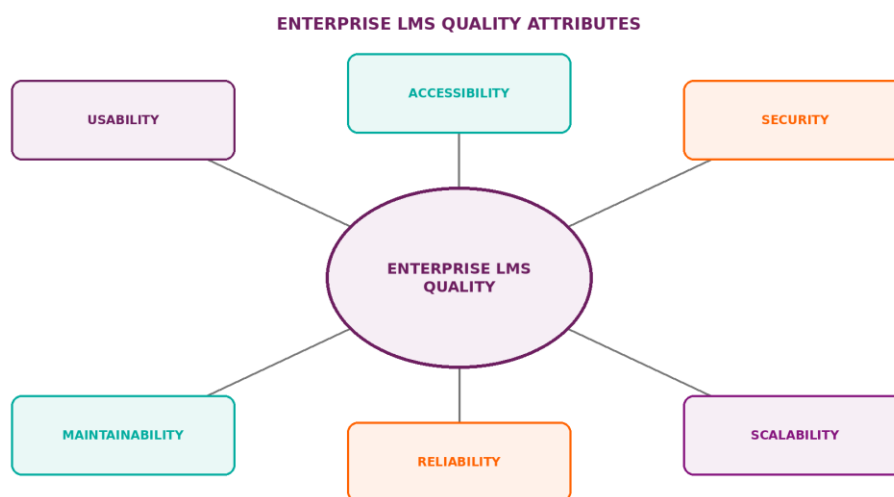


Figure 2.2: Quality attributes derived from the reviewed literature

2.13 Synthesis of the Reviewed Approaches

The literature points to a practical balance: role-focused workflows make a system easier to use; reusable components and stable API contracts help engineers maintain it; relational constraints protect data; and identity, privacy, and testing must be treated as core requirements rather than late additions.

Table 2.2: Literature insights and project implications

Reviewed area	Implication for the LMS
LMS adoption	Prioritize clear tasks, quality content and useful feedback
User experience	Use role-specific navigation and responsive, accessible controls
Web architecture	Keep client requests behind a stable service layer and gateway
Assessment	Separate automatic scoring from human evaluation and AI suggestions
Security and data	Enforce identity, tenant scope and schema rules on the server

2.14 Research Gap and Project Positioning

The LMS concept is established, and this report does not claim that the project invented it. The project's value is in applying that concept to a connected enterprise workflow: an administrator prepares courses and batches, a trainer assigns and evaluates learning, and a student completes activities whose results return to the relevant views. The report also distinguishes this working prototype from a production platform with hardened security and operations.

2.15 Chapter Summary

The reviewed work supports centralized learning records, role-oriented interfaces, accessible interaction, modular services, relational persistence, assessment feedback, and responsible analytics. It also highlights the need to validate security and usability claims against actual behavior. These ideas inform the technology and system choices discussed next.

ORGANIZATION / TOOLS & TECHNOLOGY OVERVIEW

3.1 Chapter Introduction

This chapter describes the internship setting and the technology visible in the current repository. Version and architecture statements are based on project configuration and active code, not older planning documents.

3.2 Organization Overview: Xebia

Xebia is a global technology consulting and software engineering organization. Its public material covers digital transformation, software engineering, cloud, data and AI, platform engineering, and professional training [1], [2]. The company's learning focus makes an LMS a relevant internship project, but the report treats this as an academic project developed in the internship context—not as Xebia's production LMS.

3.3 Internship Environment and Team Role

The two-month placement ran from 1 June to 31 July 2026 in remote mode. I worked as a Product Intern and Team Lead, with frontend design and development as my main technical contribution. Remote collaboration made consistent component patterns, API expectations, and regular integration review especially important.

3.4 Overall Technology Architecture

The browser hosts a React application. Its central API module sends requests to a Spring Cloud Gateway, which forwards them to five domain services. Those services use Spring controllers, service classes, repositories, and JPA entities to work with PostgreSQL. Docker and cloud configuration describe local and hosted deployment.

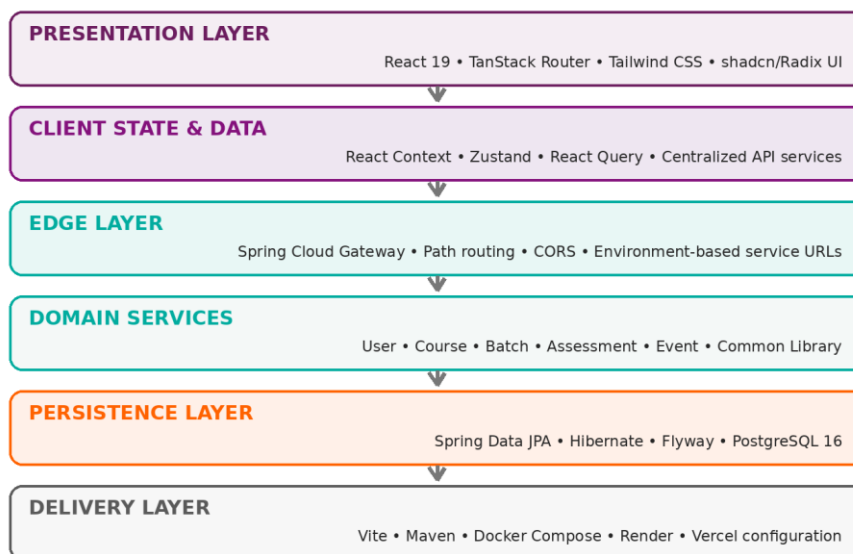


Figure 3.1: Layered technology stack of the Xebia LMS

3.5 Frontend Framework and Build Environment

3.5.1 React 19

React 19.2.0 is the main interface library. Components are used for layouts, navigation, course cards, forms, tables, assessment screens, charts, and feedback states. Most application source is JavaScript/JSX.

3.5.2 TanStack Start, Router, and Query

TanStack Start 1.167.50 and Router 1.168.25 provide the application and file-based route structure. React Query 5.83.0 handles selected server data such as courses and progress, keeping asynchronous loading separate from temporary UI state.

3.5.3 Vite and Nitro

Vite 8.0.16 supports the frontend build and local development proxy. Nitro selects a Vercel preset in the hosted frontend environment and a Node server preset elsewhere. The API base URL is selected through configuration rather than hard-coded into each page.

Table 3.1: Key technologies and their roles

Technology	Use in the project
React 19.2.0; TanStack Router 1.168.25	Portal screens, nested layouts and resource routes
TanStack Start 1.167.50; Query 5.83.0	Application framework and selected server-state requests
Tailwind CSS 4.2.1; Radix-based UI	Responsive styling and reusable accessible controls
Zustand 5.0.14; React Context	Admin shell state and shared LMS domain state
Recharts 2.15.4; Monaco Editor 4.7.0	Analytics visualizations and coding-assessment editor
Spring Boot; Spring Cloud Gateway	REST services and API routing
PostgreSQL 16; JPA/Hibernate	Relational storage and Java persistence
Docker; Vercel; Render	Local service topology and documented hosted deployment

3.6 Styling and User Interface Technologies

3.6.1 Tailwind CSS and Design Tokens

Tailwind CSS 4.2.1 supplies responsive layout, spacing, color, and dark-mode utilities. Shared purple, teal, orange, and neutral tokens give the portals a common visual identity. Reusable components help keep long utility-class combinations manageable.

3.6.2 shadcn/ui and Radix UI

The interface includes shadcn-style components based on Radix primitives for dialogs, menus, selects, tabs, tooltips, and other controls. These provide consistent behavior while allowing the project to control the visual treatment.

3.6.3 Icons, Motion, and Notifications

Lucide React supplies icons; Framer Motion supports selected transitions; and Sonner or custom toast messages report actions such as saving, publishing, submitting, and registering. Motion is used as feedback, not as a substitute for clear status information.

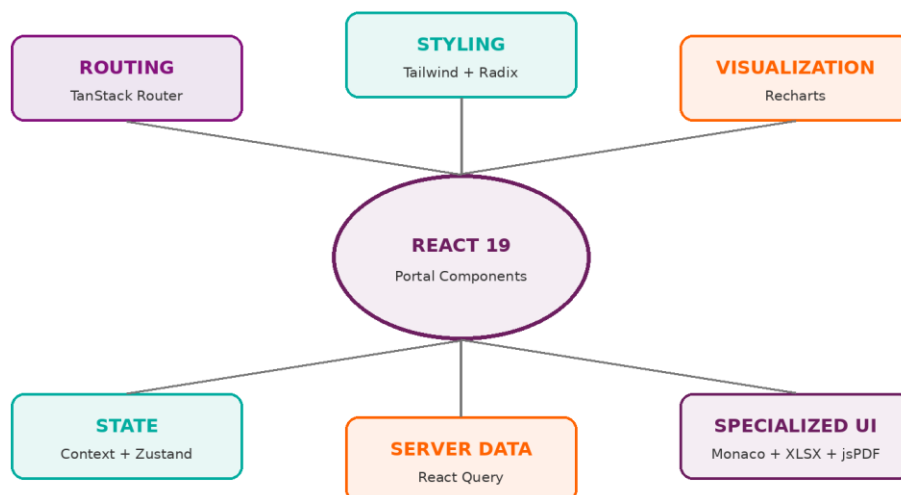


Figure 3.2: Frontend technology ecosystem and responsibilities

3.7 Frontend State and Data Management

3.7.1 LMSContext

LMSContext holds shared users, batches, assessments, submissions, notifications, current-user data, and related actions. Pages use it to present the same activity consistently across portals. Some cached values are stored in localStorage for the demonstration flow.

3.7.2 Zustand Administrative Store

Zustand holds administrative shell state such as sidebar behavior, navigation, breadcrumbs, modals, toasts, and selected dashboard values. This is kept separate from the core learning records.

3.7.3 Centralized API Service

The API module applies the base URL, JSON handling, and temporary identity and tenant headers, then exposes named client methods. This keeps individual pages from recreating request and response logic.

3.8 Specialized Frontend Libraries

Recharts powers assessment, batch, course, and learner visualizations. Monaco provides the coding workspace. React Hook Form and Zod support structured forms and validation; XLSX

exports evaluation data; and jsPDF/html2canvas support printable output. Cloudinary helps normalize media references. These libraries extend the prototype, while execution and AI services still require production security and operational controls.

3.9 Backend Language, Framework, and Build Tools

3.9.1 Java Runtime and Compilation Target

The root Maven build targets Java 17, while Docker build and runtime images use JDK/JRE 21. The report therefore distinguishes the compilation target from the container runtime.

3.9.2 Spring Boot and Spring Cloud

Gateway, common library, and Course Service inherit Spring Boot 3.3.6. User, Batch, Assessment, and Event services declare Spring Boot 4.1.0 independently. Spring Cloud uses the 2023.0.4 line in the parent-managed configuration. The repository has mixed versions, which should be aligned and verified before production.

3.9.3 Maven and Lombok

Maven 3.9.6 builds the backend modules and resolves dependencies. Lombok reduces repeated Java model code in several services.

3.10 Backend Service Technologies

3.10.1 Spring Web and REST Controllers

Spring Web controllers define the HTTP routes and receive request data. Service classes hold workflow logic so that page concerns do not need to be repeated in endpoint code.

3.10.2 Spring Data JPA and Hibernate

Repositories and JPA entities connect service operations to PostgreSQL. Hibernate manages entity relationships, while selected service methods apply additional ownership, enrollment, allocation, assessment, and event rules.

3.10.3 Validation and Security Scaffolding

Jakarta Validation supports selected request DTOs. JWT, tenant, and permission classes exist in the common library, but they are not connected to the active login and Gateway flow.

3.11 Microservices and Runtime Ports

The deployable backend contains one Gateway and five domain services. The common library is shared code, not a separate running service.

Table 3.2: Backend services and local container ports

Component	Port	Responsibility
API Gateway	8080	Frontend-facing route and CORS boundary
User Service	8081	User records and role-filtered reads
Course Service	8084	Categories, courses, curriculum and progress
Batch Service	8085	Batches and trainer allocations
Assessment Service	8086	Assessments, submissions, drafts and analytics
Event Service	8087	Events and learner registrations
Common library	—	Shared domain and security scaffolding; not deployable

3.12 Database and Schema Technologies

3.12.1 PostgreSQL 16

PostgreSQL stores the main learning records. The current model represents 17 primary JPA entity tables, three Hibernate element-collection tables, and one legacy `course_assignments` table, for 21 physical tables in total.

3.12.2 Flyway and Hibernate Schema Management

Course Service uses Flyway migrations in the course schema and Hibernate validation. Several other services use Hibernate ddl-auto update in the default schema. This mixed approach reflects the current code and is less reproducible than a single reviewed migration policy.

3.12.3 Redis and In-Memory Draft Storage

Redis 7 is configured in the local topology, but `AssessmentCacheService` currently stores drafts in a `ConcurrentHashMap`. Drafts are therefore process-local and are not Redis-backed.

3.13 External Services and AI-Related Tools

Cloudinary configuration supports media references for course content. AI-related calls support assessment description and feedback features through environment-based keys. These integrations need server-side secret management, validation, cost controls, and human review before production use.

3.14 Development Quality and Configuration Tools

ESLint and Prettier provide source checks and formatting. Environment variables hold service URLs, database credentials, Redis settings, and AI keys; example files document names without storing secrets. This separation is necessary because local Docker service names differ from hosted URLs.

3.15 Containerization and Deployment

Dockerfiles build Java services in stages, and Docker Compose defines PostgreSQL, Redis, the Gateway, and domain services for local integration. Render configuration describes backend deployments and managed PostgreSQL; the frontend uses Vercel-aware Nitro configuration. Hikari pool limits are set for constrained cloud database connections.

3.16 Technology Selection Rationale

The stack was chosen to support a large interface, clear service boundaries, relational data, and a practical deployment path. It also has trade-offs: the shared context is broad, microservices add operating overhead, schema strategies differ, and demonstration authentication is not suitable for real access control. Recognizing those limits is part of an accurate technology review.

Table 3.3: Selected tools and current implementation notes

Tool or capability	Current use or boundary
Cloudinary	Media URL support depends on environment configuration
AI service	Assistance is available; suggestions require human review
Redis	Configured infrastructure; active drafts use process memory
JWT / IAM	Scaffolding exists; active email-based demo login is separate
Vercel and Render	Deployment targets documented in project configuration

3.17 Chapter Summary

The project combines a React frontend with a gateway-plus-five-services backend and PostgreSQL persistence. The technology choices support a broad prototype, while mixed framework versions, schema management, authentication, and draft storage identify concrete areas for consolidation.

METHODOLOGY / SYSTEM DESIGN

4.1 Chapter Introduction

The design had to connect three user groups, a broad route hierarchy, five backend domains, and several forms of learning activity. The following sections describe the iterative development approach and the main decisions made while organizing those parts.

4.2 Development Methodology

The team used an iterative, feature-oriented approach rather than a formally documented Scrum process. Work was grouped around areas such as layouts, courses, batches, assessments, events, analytics, and deployment. For each area, the team clarified the user task, built a screen and data flow, connected available services, reviewed the integrated result, and refined inconsistencies.

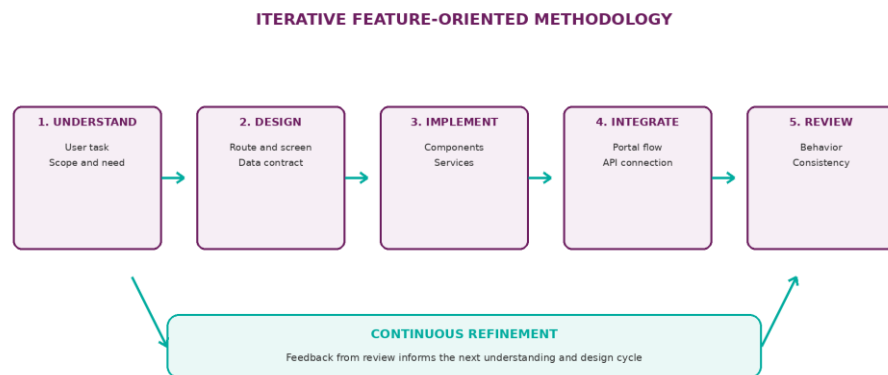


Figure 4.1: Iterative feature-oriented development cycle

4.2.1 Practical Work Phases

The work moved from the shared application shell to learning workflows, administrative tools, cross-module integration, and deployment. Some activities overlapped as requirements and API behavior became clearer.

Table 4.1: Development phases and representative outputs

Phase	Representative output
Foundation	Portal layouts, demonstration login, shared context and navigation
Learning workflows	Courses, batches, assessments, attempts, evaluation and results
Administration	Curriculum, allocations, events and analytics pages
Integration	Consistent API paths, routing, CORS and cross-portal data flow
Delivery	Docker topology, cloud configuration and API validation

4.3 Requirements Analysis

Requirements were treated as complete user tasks instead of isolated page names. For example, assessment management includes trainer authoring, publishing to batches, student attempts, submission, evaluation, and result visibility. This end-to-end view helped the team find missing connections between screens.

4.3.1 Functional Requirements

- Provide separate Admin, Trainer, and Student portals with role-relevant navigation.
- Create and organize courses, curriculum, batches, trainer allocations, assessments, and events.
- Record enrollment, progress, attempts, answers, evaluation, registration, and results.
- Present dashboards, filters, reports, exports, notifications, and certificates where supported.

4.3.2 Non-Functional Requirements

Usability, maintainability, responsiveness, data integrity, configuration, and security were considered during design. The system addresses some of these through shared components, service boundaries, relational constraints, and environment settings; production-grade authorization, observability, and accessibility testing remain incomplete.

Table 4.2: Core quality requirements and current response

Quality concern	Design response	Remaining limitation
Usability	Role-specific portals and consistent controls	Large feature surface needs user validation
Maintainability	Reusable components and service layers	Some contexts and pages remain large
Data integrity	JPA relationships, validation and service rules	Cross-service checks and migrations vary

Quality concern	Design response	Remaining limitation
Security	Gateway boundary and JWT/tenant scaffolding	Active authentication is demonstrative
Responsiveness	Flexible grids and mobile navigation	Dense tables and coding tools favor desktop

4.4 Actors and Use-Case Model

The main actors are Administrator, Trainer, and Student. PostgreSQL, media storage, and AI endpoints support selected workflows but are not human portal roles. The model reflects what the current application actually exposes; manager-oriented names in older folders do not represent an active fourth portal.

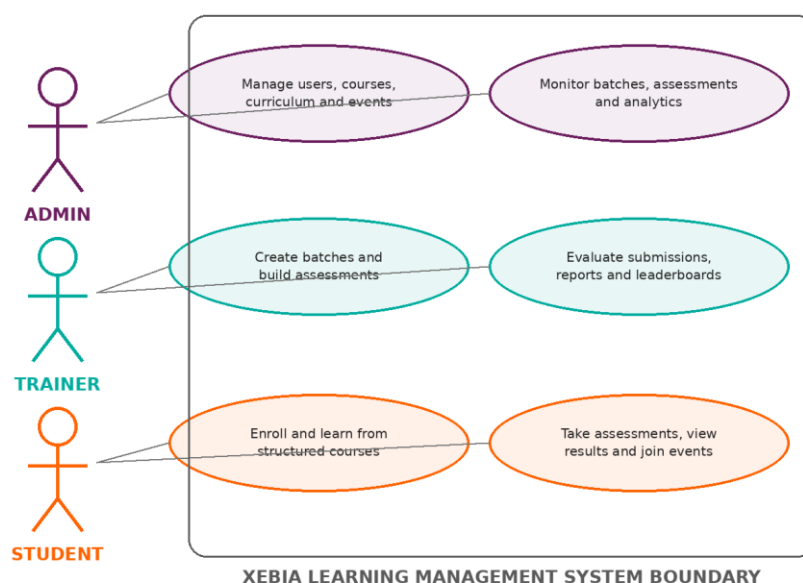


Figure 4.2: Principal actors and high-level use cases

4.5 Overall System Architecture

The browser sends requests through one API base URL. The Gateway matches the path and forwards it to the owning service; controllers receive HTTP data, services apply rules, repositories access PostgreSQL, and responses return through the Gateway. Course routes use a different prefix convention from /api/v1 routes, so the frontend centralizes those paths in its API module.

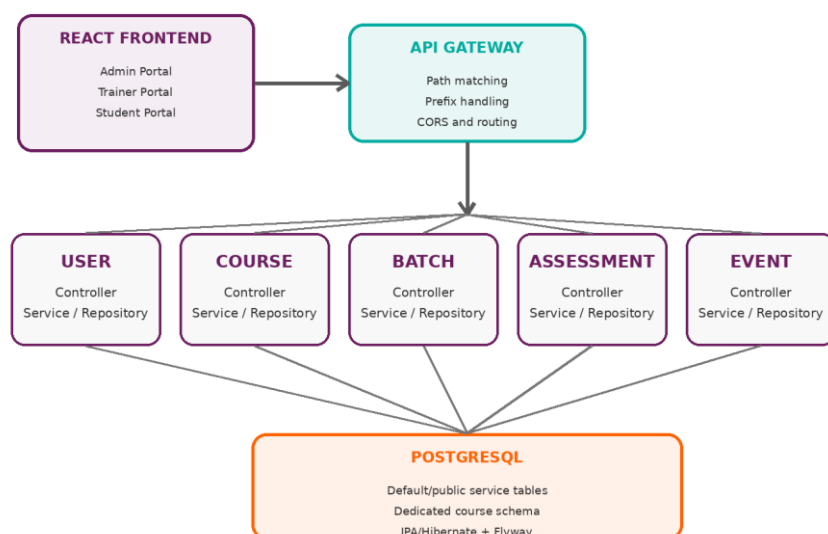


Figure 4.3: Overall frontend-gateway-service-database architecture

4.6 Frontend System Design

4.6.1 Root and Portal Layouts

The root route provides shared context and query providers. Student and Trainer layouts inspect the current session and role; the Admin layout provides a separate shell but does not apply equivalent backend-secured identity checks. Shared layout components supply navigation, headers, mobile behavior, content outlets, and toast feedback.

4.6.2 Route and Page Organization

Routes are grouped by portal and feature, with resource identifiers in dynamic paths. The source contains 73 route definitions; 65 are functional screens after structural, redirect, and legacy entries are excluded. Keeping the generated route tree free of obsolete definitions remains a maintenance task.

4.6.3 Component Layers

General controls sit below shared layouts and LMS-specific components. Pages combine those pieces into tasks such as editing a course, reviewing a submission, or registering for an event. This keeps common interface behavior reusable without forcing every feature into one generic component.

4.6.4 State and Data Flow

LMSContext holds shared domain data, Zustand manages selected administrative shell state, and React Query handles some server-backed course data. The combination reflects different lifecycles: temporary form state, shared client state, and asynchronous server state.

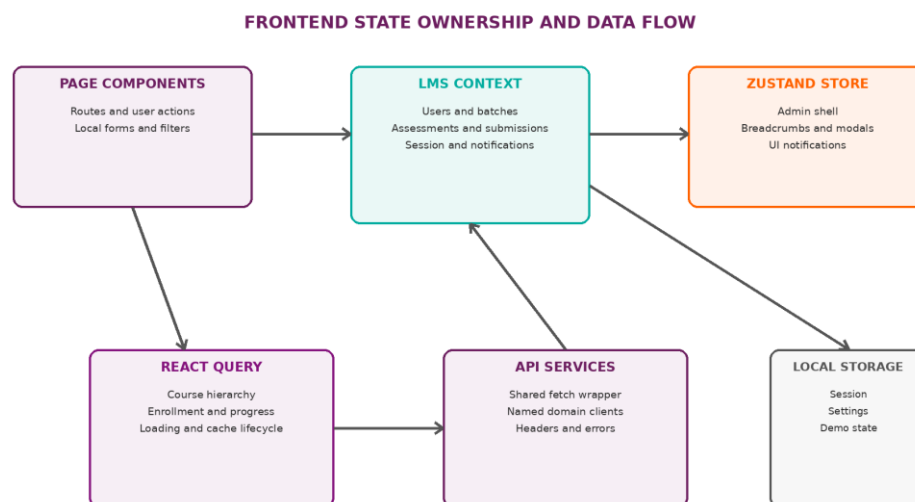


Figure 4.4: Frontend state ownership and data flow

4.7 Backend Service Design

The five domain services follow a controller–service–repository pattern. Controllers define HTTP operations, services apply validation and business rules, repositories manage persistence, and entities or DTOs represent stored and returned data. The boundary is practical for the current scale, although not every service uses DTOs and schema migrations uniformly.

Table 4.3: Backend service boundaries and endpoint mappings

Service	Mappings	Primary responsibility
User	5	User records and role-based reads
Batch	18	Batch management and trainer allocations
Assessment	20	Assessments, submissions and analytics
Course	26	Categories, curriculum, enrollment and progress
Event	11	Events and registrations

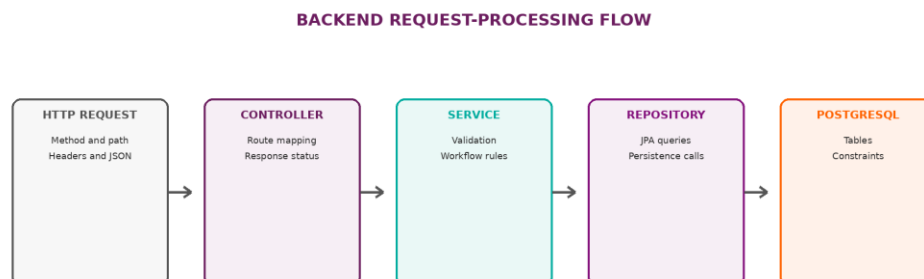


Figure 4.5: Common backend request-processing layers

4.8 API and Request-Flow Design

A shared fetch wrapper applies the API base URL, JSON headers, and temporary tenant and user identifiers. Named service methods specify resource paths and request bodies; the wrapper parses responses and reports useful errors. In event registration, for example, the page sends a request through the Gateway, Event Service checks active status, deadline, duplicate registration, and capacity, then returns a registration result for the interface to display.

4.9 Database Design

The current model contains 17 entity tables, three element-collection tables, and one legacy migration table. Course data has the strongest relational hierarchy: categories, courses, modules, submodules, content, enrollment, and progress. Other services own users, batches and allocations, assessments and submissions, and events and registrations. Cross-service identifiers are not all database foreign keys; validation across those boundaries is handled by service logic or remains future work.

Figure 4.6 gives the domain view; Figures 4.7 and 4.8 show the detailed course and operational schemas.

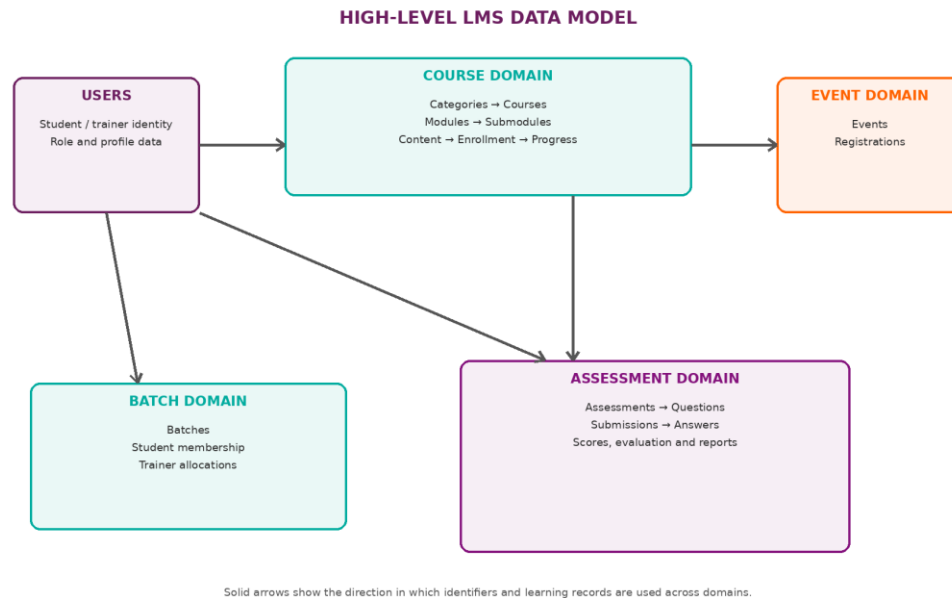


Figure 4.6: High-level entity groups and relationships

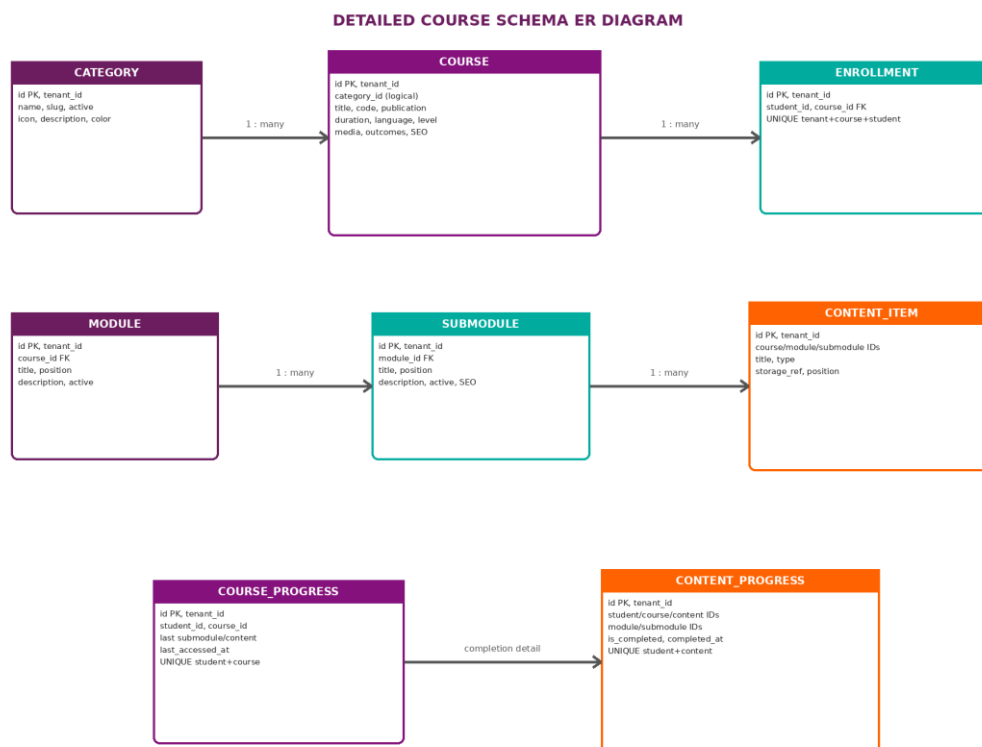
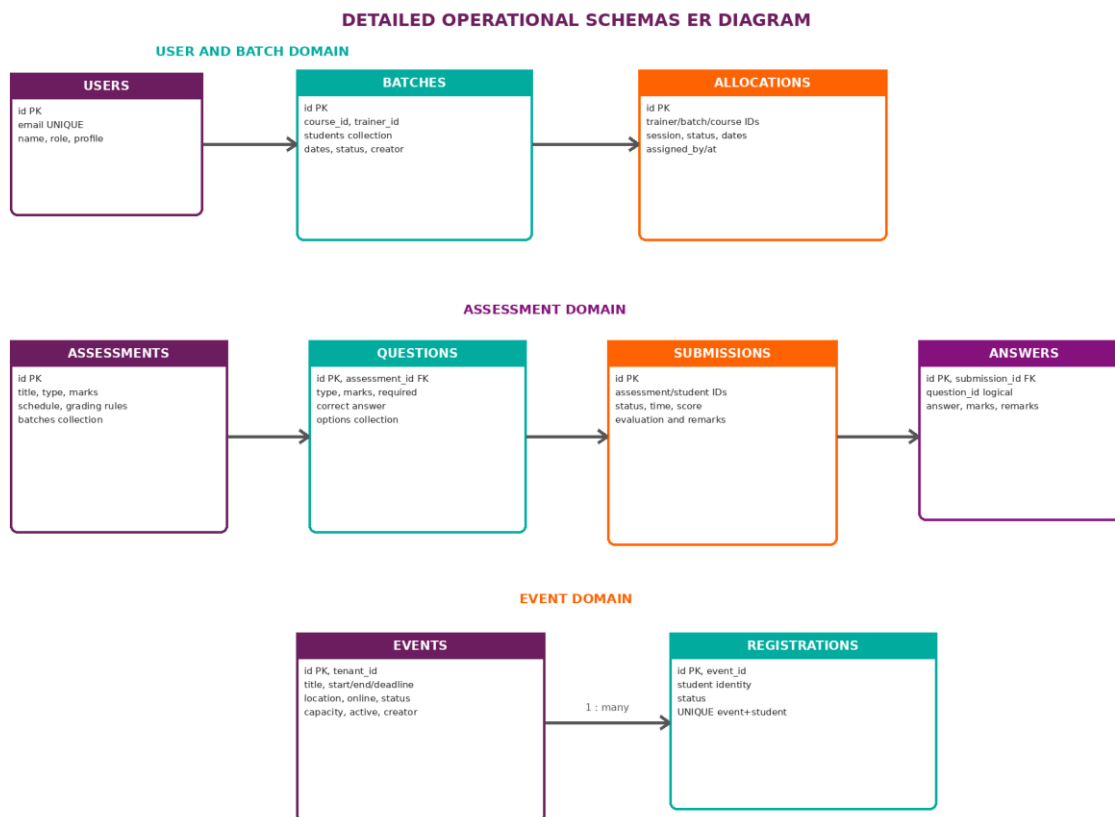


Figure 4.7: Detailed ER diagram of the Course Service schema



Cross-service student, trainer, course and batch references are logical identifiers; collection tables are generated by Hibernate.

Figure 4.8: Detailed ER diagram of User, Batch, Assessment, and Event persistence

4.10 Authentication and Tenant Design

In the active demo flow, users are loaded from the User Service and Trainer or Student login matches an email. Session data is stored in localStorage; Admin selection bypasses user lookup. JWT utilities, tenant filters, and permission helpers exist but are not wired into a verified gateway-to-service flow. Production work must validate identity server-side and derive tenant scope from trusted claims, not caller-supplied headers.

4.11 Core Domain Workflow Design

4.11.1 Course and Curriculum Workflow

Administrators create a category and course, then organize modules, submodules, and content items. Students enroll, open the hierarchy, record access, and complete learning items. The service returns progress summaries for the learner interface.

4.11.2 Batch and Allocation Workflow

A trainer creates a batch and selects an existing course. Administrators then allocate eligible trainers and courses through a guided flow. Service rules check required fields, active duplicates, and the batch-creator condition.

4.11.3 Assessment Workflow

Trainers configure assessment details and questions, assign batches, and publish. Students attempt and submit; objective items are scored automatically where supported, while descriptive responses can be reviewed manually. Evaluation and publication return results to student and administrative views.

4.11.4 Event Workflow

Administrators create events with dates, a registration deadline, capacity, and status. Students register or cancel; service checks cover date order, duplicate registration, deadlines, and capacity. Deletion is soft so that event records are not simply removed.

4.12 Analytics Design

The system combines service-provided summaries with frontend calculations from shared domain data. Views include learner activity, assessment results, batch performance, course progress, and administrative trends. These calculations work for prototype-scale data; larger deployments should use indexed server-side aggregations.

4.13 Error Handling and Validation Design

The client reports non-success responses, and pages check common conditions such as required fields, missing answers, and destructive-action confirmation. Service and database checks provide a second and third layer. The code is not uniform across every domain, so server-side validation and error formats should be standardized.

4.14 Deployment Architecture

Docker Compose supports a local stack with PostgreSQL, Redis, the Gateway, and five services. The documented hosted arrangement uses Vercel for the frontend and separate Render services

for the Gateway and domains, with managed PostgreSQL. Environment variables connect the layers.

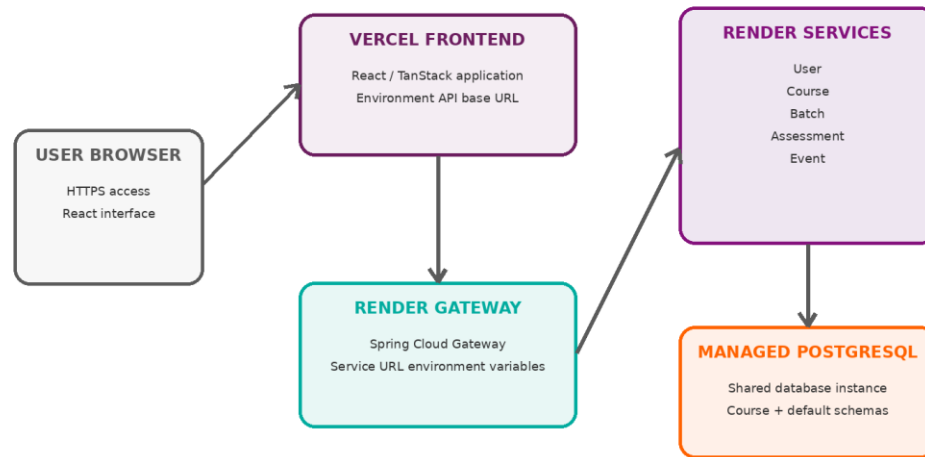


Figure 4.9: Cloud deployment topology

4.15 Key Design Decisions and Trade-offs

The choices below reflect prototype priorities as well as costs that should be addressed before production.

Table 4.4: Design decisions and trade-offs

Decision	Reason	Trade-off
Three role portals	Keep tasks relevant to each user	Shared behavior needs cross-portal consistency
Central API module	Standardize paths and errors	Path conventions still differ by service
Gateway and domain services	Separate service responsibilities	Adds deployment and diagnosis overhead
PostgreSQL with JPA	Model related learning records	Queries and migrations require review
Demonstration login	Support role workflows during development	Not suitable for real access control

4.16 Chapter Summary

The design connects role-based workflows to services and data while making security, validation, and deployment limits clear.

IMPLEMENTATION

5.1 Chapter Introduction

Implementation turned the architecture into three portal experiences backed by a shared frontend service layer, one Gateway, five domain services, and PostgreSQL. This chapter concentrates on the user-visible work and the main links between interface actions and backend behavior.

5.2 Repository and Source Organization

The React application lives at the repository root, with routes under `src/routes` and larger pages and components organized under `src`. The API module and `LMSContext` connect screens to shared state and service calls. Backend applications use `controller`, `service`, `repository`, `entity/DTO`, and `configuration` packages. This structure makes it possible to trace a page action into a client method and then into the owning service.

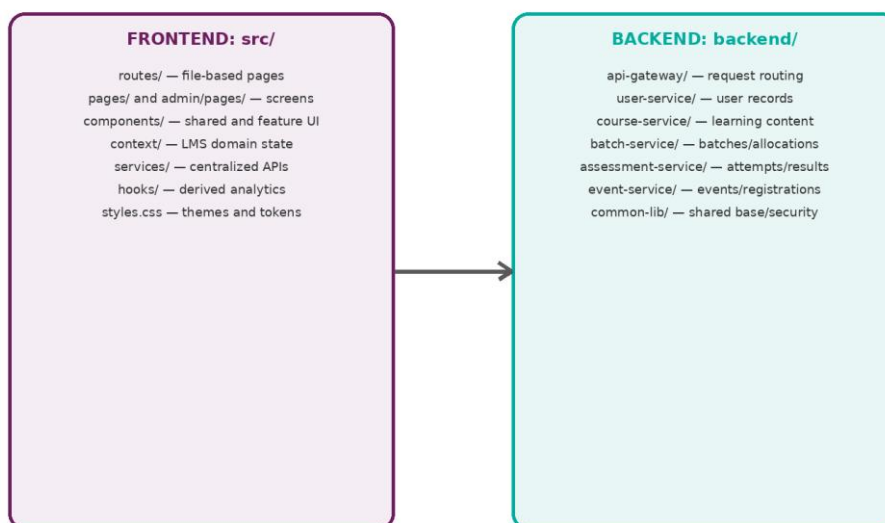


Figure 5.1: Principal frontend and backend source organization

5.3 Application Bootstrap, Session, and Shared Layout

At startup, the frontend initializes providers, loads or restores selected cached data, and requests users, batches, assessments, and submissions. The login screen demonstrates role selection. Trainer and Student login match an email to loaded users; their layouts check the current role. Admin entry is a direct demonstration path. Shared layout components provide the sidebar, header, content area, mobile controls, and notifications.

5.4 Admin Portal Implementation

5.4.1 Dashboard, Categories, and Users

The Admin dashboard derives summary cards from users, courses, categories, batches, and assessments. Category screens support listing and maintenance; user screens support role-based browsing, creation, and deletion. These records feed later course, batch, and reporting workflows.

5.4.2 Course and Curriculum Management

Course forms organize descriptive details, category, visibility, difficulty, duration, outcomes, prerequisites, and media information into sections. Curriculum is hierarchical: modules contain

submodules, which contain typed content items. The student viewer receives that hierarchy as a nested response, so the authoring structure becomes the learner's navigation.

5.4.3 Batch Management and Trainer Allocation

Administrators can inspect batches, allocations, workload, and analytics. The allocation wizard selects a batch, trainer, and course, then submits a reviewed assignment. The matrix resolves identifiers into names and supports filtering and export. Service checks help prevent duplicate active allocations and enforce the configured batch-creator rule.

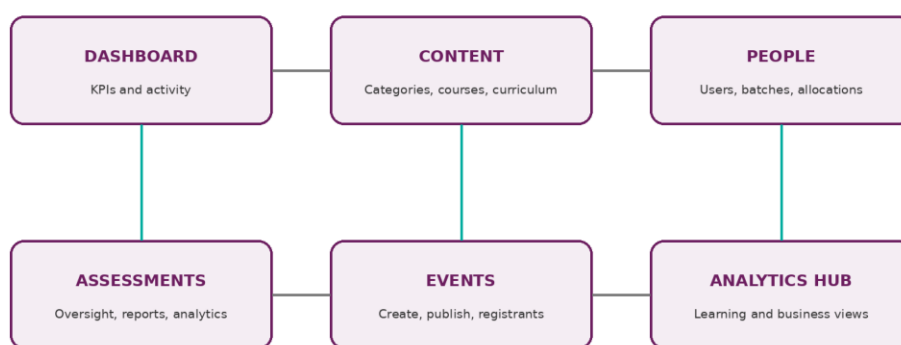


Figure 5.2: Admin Portal implementation areas

5.4.4 Administrative Assessment Management

Administrative assessment pages combine assessment records with summary measures such as attempts, average score, pass rate, and submission activity. Detail views present learner outcomes and trends for oversight.

5.4.5 Event and Analytics Hub Implementation

Event administration supports draft or published state, date and capacity information, registrant inspection, and soft deletion. The Analytics Hub groups several learning and workforce views under shared filters and visual conventions.

5.5 Trainer Portal Implementation

5.5.1 Trainer Dashboard and Batch Ownership

The Trainer dashboard filters batches and assessments by the current trainer and highlights pending evaluation. Trainers can create batches, select courses, and review their assigned activity.

5.5.2 Assessment Builder

The builder collects title, schedule, duration, pass marks, attempt limits, question order, batch selection, and publication status. Question editors adapt to the answer type; temporary browser IDs are removed before new questions are sent to the service.

Table 5.1: Assessment question types in the interface

Type	Learner response	Evaluation approach
MCQ / True–False	Select one option	Automatic scoring
Multi-select	Select several options	Automatic exact-set comparison
Short answer / paragraph	Enter text	Trainer review or AI-assisted suggestion
File upload	Attach file metadata in prototype	Manual review
Coding	Submit source and language	AI-assisted simulation or trainer review

5.5.3 Evaluation, Leaderboard, and Reports

The evaluation workspace places a submission queue beside the learner response and grading controls. Trainers can assign marks, add remarks, provide overall feedback, and publish results. Spreadsheet export and leaderboard/report views support follow-up without creating a separate performance record for each portal.

5.6 Student Portal Implementation

5.6.1 Dashboard, Batches, and Profile

The student dashboard summarizes current courses, batches, assessments, notifications, and performance. Batch views filter membership to the current student, while profile components display identity and selected progress information.

5.6.2 Course Discovery, Enrollment, and Learning

Students can browse, filter, and enroll in courses. The course viewer displays modules, submodules, and content blocks, with different renderers for formats such as video, text, links, documents, images, and quizzes. Access and completion actions update course progress.

5.6.3 Objective Assessment Attempt

A timed assessment can resume an in-progress submission, restore answers, and display question state such as answered or flagged. The submission flow checks required responses, scores supported objective items, applies configured negative marking, and routes descriptive responses for review.

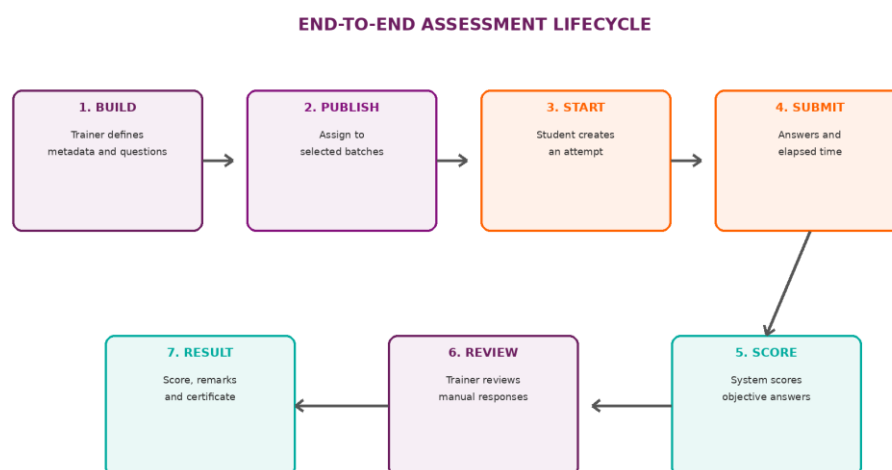


Figure 5.3: End-to-end assessment lifecycle

5.6.4 Coding Assessment Workspace

The coding workspace combines a problem statement, Monaco editor, language selection, starter templates, test-case views, timer, and submission controls. Drafts are kept in localStorage. This is a coding-assessment interface, not an isolated production execution environment.

5.6.5 Results, Certificates, Events, and Feedback

Result pages show scores, evaluation status, and answer review; certificate screens create a printable learner artifact. Event pages expose registration and cancellation, while trainer and

admin views can inspect participation. Feedback and notifications are primarily frontend-oriented because there is no complete dedicated persistence service for them.

5.7 Frontend API and Context Implementation

The shared fetch wrapper handles headers, response parsing, and errors. Named client methods cover courses, users, batches, assessments, submissions, allocations, events, and progress. LMSContext also coordinates selected actions such as attempts, evaluation, notifications, and leaderboards. Its breadth makes it a useful future refactoring target as more authority moves to backend services.

5.8 Backend Implementation

5.8.1 API Gateway

The Gateway defines path routes and uses environment-driven service destinations. It centralizes CORS and exposes actuator endpoints. JWT properties are present, but an active authentication filter is not registered.

5.8.2 User and Batch Services

User Service supports reads, single and bulk creation, and deletion. Batch Service manages batches and trainer allocations, including summaries and filters for trainer, batch, and course.

5.8.3 Course Service

Course Service manages categories, courses, curriculum hierarchy, enrollment, and progress. It applies tenant context to selected operations and returns hierarchy and progress DTOs for the frontend.

5.8.4 Assessment and Event Services

Assessment Service manages assessments, questions, submissions, and selected aggregates; temporary question IDs are sanitized before persistence. Event Service validates date order, registration deadlines, duplicate participation, and capacity. Assessment drafts remain in process memory despite Redis configuration.

Across services, selected course changes check tenant ownership; duplicate enrollment and allocation are prevented; progress is derived from completed content; temporary question IDs are sanitized; and event dates, deadlines, duplicates, and capacity are validated.

5.9 Database Entity Implementation

The services persist users; batches and allocations; courses and curriculum; assessments, questions, submissions, and answers; and events and registrations. Element collections represent batch membership, assessment batch IDs, and question options. Several links between services use identifiers instead of database foreign keys because each domain owns its records.

5.10 Representative End-to-End Implementation Traces

5.10.1 Course Enrollment and Progress

A student enrolls through Course Service, then records lesson completion; the service updates progress for the course viewer.

5.10.2 Assessment Submission and Evaluation

A student submits an attempt; supported objective items are scored, and a trainer can update marks and remarks before the result is shown.

5.10.3 Event Registration

Event Service checks a student's registration request, stores the status, and exposes it to the administration view. Cancellation updates that status.

5.11 Personal Implementation Contribution

My main contribution was frontend design and development: portal navigation, shared layouts, responsive behavior, reusable patterns, and API-linked interactions. As Team Lead, I coordinated integration and helped keep modules consistent. Backend services were a team effort.

5.12 Chapter Summary

The three portals connect course, batch, assessment, event, evaluation, and reporting workflows while retaining clear prototype boundaries.

RESULTS AND DISCUSSION

6.1 Chapter Introduction

The result is an integrated LMS prototype rather than a set of unrelated page demonstrations. This chapter summarizes what was delivered and separates implemented behavior from future-facing configuration.

6.2 Overall Project Outcome

The repository contains three role-focused portals, 65 functional routed screens, one API Gateway, five domain services, a shared library, 80 controller endpoint mappings, and a model of 21 physical database tables. More important than these counts is the continuity between roles: an administrator can set up a course or event, a trainer can use the course in a batch and publish an assessment, and a student can complete the activity whose result becomes visible in trainer and administrative views.

Table 6.1: Project inventory and implementation scale

Measure	Observed result
Frontend routes	73 definitions; 65 functional screens
Backend applications	One Gateway and five domain services
Shared backend code	One common library; not a deployable service
Controller mappings	80 across User, Batch, Assessment, Course and Event
Persistence model	17 entity tables, 3 collection tables, 1 legacy table
Primary portals	Admin, Trainer and Student

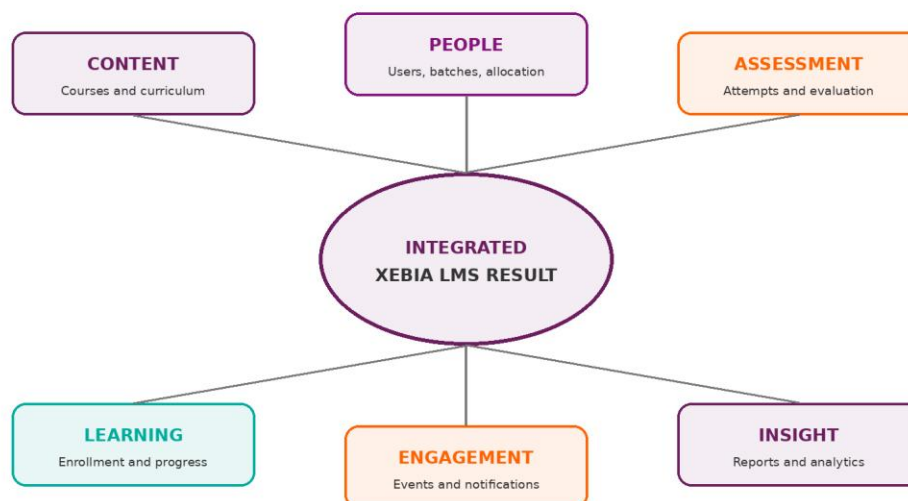


Figure 6.1: Integrated functional outcomes of the project

6.3 Results Against Project Objectives

The core objectives were substantially met in the implemented prototype. The table summarizes the evidence and distinguishes areas that need further work.

Table 6.2: Outcome against project objectives

Objective	Result
Role-based LMS	Admin, Trainer and Student workflows share one application
Structured learning	Course hierarchy, enrollment and progress are implemented
Assessment lifecycle	Authoring, attempts, scoring, manual review and results are connected
Events and coordination	Batch allocation and learner registration are represented
Analytics	Dashboards derive and present learning and assessment data
Production identity	Not achieved; current login is demonstrative

6.4 Portal-Wise Results

6.4.1 Admin Portal Result

The Admin Portal brings course structure, users, batches, allocations, events, assessment oversight, and analytics into one navigation system. Its value is the connection between setup and oversight, not simply the number of forms.

6.4.2 Trainer Portal Result

The Trainer Portal supports batch activity, assessment creation, publishing, evaluation, reports, and leaderboards. Creator-based filtering helps keep the assessment builder focused on the trainer's own work.

6.4.3 Student Portal Result

The Student Portal connects course discovery, lesson progress, events, timed assessments, coding activities, results, and certificates. Status indicators and feedback help students understand the current step without knowing which service processed it.

6.5 Frontend and User-Experience Results

Shared colors, navigation, cards, forms, status badges, charts, and feedback controls give the large interface a common identity. Responsive grids, mobile navigation, wrapped filters, and overflow handling improve use on smaller screens. The result is a broad prototype, not a formally certified accessibility implementation; dense tables and the coding workspace remain more comfortable on larger displays.

6.6 Backend and Data Results

Domain services persist the central learning records and apply rules for enrollment, progress, allocation, assessment updates, event dates, deadlines, duplicates, and capacity. Cross-service identifiers and mixed migration practices remain limitations. The draft cache is process-local even though Redis is configured.

6.7 Assessment and Analytics Results

Assessment is the most connected workflow: question configuration leads to learner attempts, objective scoring or manual evaluation, and result views. Analytics reuse submissions, batches, courses, and allocations to show performance and activity. Several analytics are calculated in the browser, which is appropriate for prototype data but should move to indexed backend queries at larger scale.

6.8 Deployment and Availability Results

The repository includes local Docker configuration and documented hosted endpoints. These URLs are reproduced from the project README; their availability depends on the lifecycle and status of the corresponding services.

Table 6.3: Deployment endpoints recorded in the project README

Component	Documented URL
Frontend	https://xebia-enterprise-lms.vercel.app
API Gateway	https://xebia-api-gateway-mritunjai.onrender.com
User Service	https://xebia-user-service-mritunjai.onrender.com
Course Service	https://xebia-course-service-mritunjai.onrender.com
Batch Service	https://xebia-batch-service-mritunjai.onrender.com
Assessment Service	https://xebia-assessment-service-mritunjai.onrender.com
Event Service	https://xebia-event-service-mritunjai.onrender.com

6.9 Documented Load-Test Results

Project notes describe a concurrent CRUD exercise with 50 requests per operation phase across five services: five services $\times$ four CRUD phases $\times$ 50 requests, or 1,000 recorded operations. The exercise was documented during project work and was not independently rerun for this report. It is a basic concurrency and burst observation—not a professional DDoS test and not evidence of DDoS protection.

	POST	GET	PUT	DELETE
USER	50 concurrent	50 concurrent	50 concurrent	50 concurrent
BATCH	50 concurrent	50 concurrent	50 concurrent	50 concurrent
ASSESSMENT	50 concurrent	50 concurrent	50 concurrent	50 concurrent
COURSE	50 concurrent	50 concurrent	50 concurrent	50 concurrent
EVENT	50 concurrent	50 concurrent	50 concurrent	50 concurrent

DOCUMENTED TOTAL: 5 services × 4 CRUD phases × 50 = 1,000 operations

Figure 6.2: Summary of the documented concurrent CRUD exercise

The team recorded connection pressure during full-list reads, uniqueness collisions in repeated user data, and event failures when required fields or date order were invalid. Pool limits and valid, timestamped test data were used during later runs. Those findings are more useful than treating the exercise as a simple pass count.

6.10 Discussion of Business and Learning Value

The platform reduces the need to switch between separate tools for content, assessments, events, allocations, and reports. It also links learning activity to evidence: course completion, assessment results, trainer review, and administrative views. Technology alone does not ensure learning quality; course material, trainer involvement, fair assessment, and learner support remain important.

6.11 Limitations Observed in the Results

- Authentication and tenant headers are suitable for demonstration only; JWT validation is not active.
- Coding evaluation is not a secure, isolated production judge; AI suggestions require review.
- Notifications, feedback, and assessment drafts are not fully persisted through dedicated services.
- Spring Boot versions and database migration strategies vary across services.
- Some analytics are calculated in the browser, and formal automated test coverage is limited.

6.12 Internship Learning Outcomes

The internship strengthened my ability to turn product expectations into navigation, data needs, actions, and feedback states. Frontend work involved more than visual styling: forms had to match API contracts, dashboards had to use shared records, and role-specific flows had to remain consistent. Leading the group also made integration a shared responsibility. Reviewing the active code against older documentation taught me to distinguish implemented behavior from configuration or scaffolding.

6.13 Chapter Summary

The main outcome is a connected, role-based LMS prototype with substantial course, batch, assessment, event, reporting, and deployment work. The project demonstrates product breadth and realistic integration while identifying security, persistence, testing, and scalability work still required.

TESTING

7.1 Testing Objective and Approach

Testing took place during feature integration, not only at final demonstration. Browser checks covered user-facing behavior, Postman checks exercised request paths and payloads, and cross-portal workflows were followed from setup through learner action and result. The focus was whether the expected state was produced and visible to the next user, not only whether a page loaded.

7.2 Test Environment and Tools

Evidence came from manual browser review, Postman, the local service topology, PostgreSQL persistence checks, and documented concurrent request scripts. The report records project testing; it does not claim formal certification or a new test run during editing.

Table 7.1: Testing coverage and evidence

Area	Coverage recorded
Frontend	Manual functional review of 65 routed screens
API	80 controller mappings checked through functional requests
Integration	Course, batch, assessment, event and user workflows across portals
Data	Create, update, read-back, uniqueness and selected cascade behavior
Responsive/security-oriented review	Reduced-width layout, CORS, headers, validation and route behavior

7.3 Page-by-Page Functional Testing

The functional screen review checked intended portal navigation, visible content, primary actions, form behavior, filters or search where available, and movement to related detail pages. Dynamic routes used representative identifiers. Layout-only, redirect-only, and legacy routes were not counted as functional screens.

7.4 Portal Workflow and Integration Testing

End-to-end checks linked the portals rather than treating them as isolated features.

Table 7.2: Representative workflow and boundary tests

Test flow	Expected behavior	Result recorded
Course: create → curriculum → enroll → complete lesson	Hierarchy and progress appear to the learner	Passed functional review
Assessment: build → publish → attempt → evaluate → result	One submission is shared across portals	Passed functional review
Event: create → register → inspect → cancel	Deadline and capacity rules are reflected	Passed functional review
Allocation: create batch → assign trainer/course	Ownership and allocation views refresh	Passed functional review
Boundary checks: duplicates, invalid dates, missing fields	Service rejects or prevents invalid state	Validated in project testing

7.5 Postman API Testing

Postman requests used the Gateway and, where needed for diagnosis, direct service URLs. Positive cases checked status codes, response shapes, IDs, and follow-up reads. Negative cases included missing values, duplicate identifiers, unknown records, and invalid event timing. The current controller inventory contains 80 mappings: User 5, Batch 18, Assessment 20, Course 26, and Event 11.

7.6 Concurrent 50-Request Burst Testing

Project notes describe 50 simultaneous requests for each CRUD phase on each of the five services. This pattern was used to observe basic concurrency, connection use, and write consistency. Fifty concurrent requests are not a distributed denial-of-service assessment and do not prove protection from an attack.

Table 7.3: Documented concurrent CRUD exercise

Service	Create	Read	Update	Delete
User	50	50	50	50
Batch	50	50	50	50
Assessment	50	50	50	50
Course	50	50	50	50
Event	50	50	50	50

7.6.1 Load-Related Observations

The exercise exposed connection pressure during full-list reads and showed why unique test data and realistic timestamps matter. Event requests had to include required fields and valid deadline/start/end order. Hikari pools were limited to ten connections per service and the Gateway HTTP pool was configured at 500; these settings manage resource use but do not create unlimited capacity.

7.7 Validation and Negative Testing

Selected negative cases included empty bulk-user input, duplicate enrollment or allocation, an unknown submission update, registration after deadline, a full event, and unanswered required quiz questions. Expected outcomes ranged from a validation response to a conflict or warning. These cases helped confirm that important rules were not only represented in the interface.

7.8 Database and Persistence Testing

Writes were followed by reads to check that generated identifiers and saved values persisted. Course hierarchy tests checked parent relationships; assessment checks covered questions and answers; and registration tests exercised uniqueness and status changes. Event deletion is soft, while some other resources use removal or cascade behavior.

7.9 Responsive, Accessibility, and Visual Testing

Reduced-width checks covered mobile navigation, card stacking, filters, tables, charts, text visibility, and theme contrast. These were practical visual checks rather than a formal WCAG audit or assistive-technology certification. Keyboard and screen-reader testing should be expanded.

7.10 Security-Oriented Testing and Observations

The team reviewed CORS behavior, headers, role-route behavior, malformed input, and burst load. A duplicate CORS-header problem was corrected by making the Gateway the primary CORS authority. The review also confirmed that frontend checks cannot replace API authorization and that Admin entry is not identity-verified.

7.11 Defects Found and Corrective Outcomes

Integration testing led to corrections in routing, navigation, request data, and assessment handling.

Table 7.4: Representative integration defects and fixes

Issue	Correction
Duplicate CORS headers	Consolidated CORS ownership at the Gateway
Admin child routes did not render	Corrected route tree and parent layout outlet
Multiple sidebar items appeared active	Changed navigation to explicit route matching
Trainer assessment list included unrelated records	Filtered by the current trainer
Temporary question IDs failed persistence	Sanitized IDs before saving

7.12 Coverage Limitations

The project has broad manual, Postman, workflow, and boundary coverage, but component tests, browser automation, service integration tests, contract tests, accessibility automation, security scanning, and long-duration load tests remain limited or absent. Cloud free-tier behavior also differs from a production environment.

7.13 Testing Conclusion

Testing confirmed that the principal pages, APIs, business rules, persistence operations, and cross-portal workflows behaved as intended during project validation. The concurrent burst exercise exposed practical infrastructure and payload issues, while integration review found defects that were corrected. Repeatable automated tests and production identity controls are the next important steps.

CONCLUSION AND FUTURE SCOPE OF WORK

8.1 Conclusion

The internship produced a full-stack LMS prototype for administrators, trainers, and students. Its main achievement is the connection between course setup, batches, trainer allocation, assessments, learner progress, events, evaluation, and reporting. The system is broad enough to demonstrate realistic learning operations, while the report remains explicit about prototype boundaries.

8.1.1 Achievement of the Internship Objectives

The project substantially met its core objectives: it delivered three role-focused portals, structured learning content, assessment attempts and evaluation, event participation, analytics, relational persistence, and deployment configuration. Current identity, draft storage, feedback, and coding execution are not production-ready; they are not presented as completed security or reliability controls.

8.1.2 Personal and Professional Learning

The Product Intern role strengthened my understanding of how user needs become screens, data contracts, and feedback states. Team leadership required checking how course, batch, and assessment modules fit together, not just whether each one worked in isolation. Leading a remote group and keeping the frontend coherent were important parts of my internship. The experience also reinforced the value of checking active code rather than assuming that every planning document is current.

Table 8.1: Final project outcomes

Outcome area	Result
Product	Integrated LMS workflows for three user roles
Frontend	65 functional screens with shared navigation and design patterns
Backend	One Gateway, five services and 80 endpoint mappings
Data	Relational records for core learning workflows

Outcome area	Result
Testing	Manual, API, workflow, boundary and burst checks recorded
Security	Scaffolding present; production authentication remains future work

8.2 Future Scope of Work

Future work should preserve the connected workflow while replacing prototype shortcuts with production controls.

8.2.1 Production Authentication and Authorization

Integrate IAM, short-lived signed tokens, server-side permission checks, trusted tenant context, and verified Admin access. The Gateway should reject caller-supplied identity headers.

8.2.2 Persistent Notifications, Feedback, and Real-Time Updates

Move notifications and feedback to persistent services and add delivery state. Real-time updates could surface published assessments, submissions, evaluation, allocations, and upcoming events.

8.2.3 Secure Coding Assessment Infrastructure

Connect the editor to an isolated judge with CPU, memory, network, and time limits. Keep AI assistance optional; deterministic test execution should decide correctness.

8.2.4 Data and Schema Governance

Use reviewed, versioned migrations across services, retire legacy tables safely, document backups, and validate cross-service identifiers through explicit contracts.

8.2.5 Redis, Caching, and Resilience

Replace process-local draft storage with durable or Redis-backed storage. Add measured timeouts, rate limits, health checks, logs, metrics, and resilience policies before making availability claims.

8.2.6 Analytics and Reporting

Move large aggregations to indexed backend queries and document metric definitions. Apply tenant and privacy rules to learner-level data and predictive features.

8.2.7 Automated Testing and Quality Engineering

Add component, end-to-end, service integration, contract, accessibility, and controlled load tests to a repeatable CI pipeline.

8.2.8 User Experience and Learning Features

Validate dense screens with users, complete keyboard and screen-reader checks, and consider learning paths, reminders, verified certificates, multilingual content, and richer learner support.

8.2.9 Codebase Consolidation and Maintainability

Remove or redirect legacy route shells, split large contexts and pages, standardize DTOs and error responses, align Spring Boot versions, and mark historical planning documents as archived.

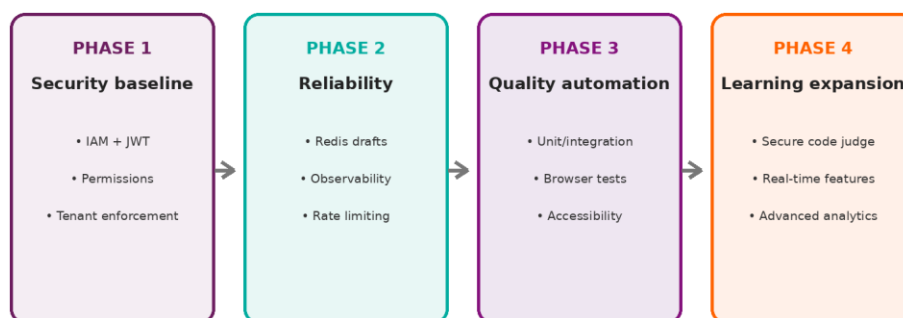


Figure 8.1: Proposed future development roadmap

Table 8.2: Prioritized future-development roadmap

Priority	Work item	Expected benefit
Critical	IAM, JWT and tenant enforcement	Verified identity and controlled access
High	Secure coding judge and automated tests	Safer assessment and repeatable confidence
High	Persistent notifications, feedback and drafts	Reliable cross-device continuity
Medium	Unified migrations and dependency versions	Reproducible builds and schemas
Medium	Backend analytics and observability	More efficient reporting and diagnosis
Ongoing	Accessibility and responsive refinement	Inclusive, validated user experience

REFERENCES

- [1] Xebia, “Our Locations,” Xebia. [Online]. Available: <https://xebia.com/locations/>. [Accessed: Aug. 11, 2026].
- [2] Xebia, “Contact Xebia—Global AI and IT Consulting Experts,” Xebia. [Online]. Available: <https://xebia.com/about-us/contact/>. [Accessed: Aug. 11, 2026].
- [3] “The Effectiveness of Learning Management System (LMS) Use in Higher Education,” International Journal of Current Educational Research, 2025. [Online]. Available: <https://journal.jcopublishing.com/index.php/ijcer/article/view/625>.
- [4] “Leveraging Learning Analytics to Improve the User Experience of Learning Management Systems in Higher Education Institutions,” Information, vol. 16, no. 5, Art. no. 419, 2025. [Online]. Available: <https://www.mdpi.com/2078-2489/16/5/419>.
- [5] Meta Platforms, Inc., “React—The Library for Web and Native User Interfaces,” React Documentation. [Online]. Available: <https://react.dev/>. [Accessed: Aug. 11, 2026].
- [6] TanStack, “TanStack Router Overview,” TanStack Documentation. [Online]. Available: <https://tanstack.com/router/latest/docs/overview>. [Accessed: Aug. 11, 2026].
- [7] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA, USA: O’Reilly Media, 2021.
- [8] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning Publications, 2018.
- [9] R. T. Fielding, “Architectural Styles and the Design of Network-Based Software Architectures,” Ph.D. dissertation, Dept. Information and Computer Science, Univ. of California, Irvine, CA, USA, 2000.
- [10] PostgreSQL Global Development Group, “PostgreSQL Documentation,” PostgreSQL. [Online]. Available: <https://www.postgresql.org/docs/current/>. [Accessed: Aug. 11, 2026].
- [11] OWASP Foundation, “Testing JSON Web Tokens,” Web Security Testing Guide. [Online]. Available: https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/10-Testing_JSON_Web_Tokens. [Accessed: Aug. 11, 2026].
- [12] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Oct. 2023. [Online]. Available: <https://www.w3.org/TR/WCAG22/>.
- [13] VMware, Inc., “Spring Cloud Gateway Reference Documentation,” Spring. [Online]. Available: <https://docs.spring.io/spring-cloud-gateway/reference/>. [Accessed: Aug. 11, 2026].
- [14] M. Singh et al., “Xebia Enterprise LMS,” GitHub repository, 2026. [Online]. Available: <https://github.com/mritunjai-prog/Xebia-Enterprise-LMS>. [Accessed: Aug. 11, 2026].

ANNEXURE - I

MONTHLY ATTENDANCE REPORT

Reserved for the signed or system-generated monthly attendance report covering the internship period from 1 June 2026 to 31 July 2026. The final evidence may be inserted as an image or scanned document without changing the chapter sequence.

Attach Monthly Attendance Report Here

ANNEXURE - II

FEEDBACK FORM FROM SUPERVISOR

Reserved for the completed supervisor feedback form bearing the appropriate signature and organization stamp. The final scanned form may be inserted in this area before submission.

Attach Signed Supervisor Feedback Form Here